

Course: Data Science and Analytics

Level: Bachelor

Program: Computer/Software Engineering

Pokhara University

Practical topics:

1. Data preprocessing: Cleaning and integrating datasets from multiple sources
2. Plotting and Inferring empirical distribution
3. Correlation Analysis
4. Applying and Interpreting Multivariate Linear Regression
5. Non-parametric Regression using Kernel methods
6. Principal Component Analysis (PCA)
7. Decision Trees for Classification
8. Logistic Regression
9. ARMA and ARIMA
10. Granger Causal Inference

Lab 1: Data preprocessing: Cleaning and integrating datasets from multiple sources

Title

Data Preprocessing: Cleaning and Integrating Datasets from Multiple Sources

Objective

- To understand the importance of data preprocessing in the data analysis pipeline
- To identify common data quality issues such as missing values, duplicates, and inconsistencies
- To apply data cleaning techniques to improve data quality
- To integrate data from multiple sources into a single, consistent dataset suitable for analysis

Software / Tools Required

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries:
 - pandas
 - numpy
 - matplotlib (optional, for inspection)

Theoretical Background (Brief)

Real-world data is often **incomplete, inconsistent, noisy, and distributed across multiple sources**.

According to the **Knowledge Discovery in Databases (KDD)** process, **Data Preprocessing** is a critical step that includes:

1. Data Cleaning

- Handling missing values
- Removing duplicate records
- Correcting data formats and inconsistencies
- Detecting and treating outliers

2. Data Integration

- Combining datasets from different sources

- Resolving schema conflicts
- Removing redundancy
- Ensuring entity consistency

Proper preprocessing improves data quality, model accuracy, and decision reliability

Dataset Description

We use **two datasets from different sources**:

Dataset 1: students_info.csv

Student_ID	Name	Age	Gender
101	Ram	20	M
102	Sita		F
103	Hari	22	Male
104	Gita	21	F
104	Gita	21	F

Dataset 2: students_marks.csv

Student_ID	Marks	Attendance
101	78	85
102	82	
103		90
105	75	80

Lab Tasks

Task 1: Load Datasets

- Load both datasets using pandas
- Display structure and summary

Task 2: Data Cleaning

- Handle missing values
- Remove duplicate records
- Standardize categorical values

Task 3: Data Integration

- Merge datasets using a common key
- Resolve missing and unmatched records

Task 4: Save Cleaned Dataset

- Export the final integrated dataset

Algorithm / Steps

1. Import required libraries
2. Load datasets from CSV files
3. Inspect data for missing values and duplicates
4. Clean data:
 - Fill or remove missing values
 - Remove duplicates
 - Standardize text formats
5. Integrate datasets using Student_ID
6. Validate the final dataset
7. Save the cleaned dataset

Program / Implementation

```
# Step 1: Import libraries
import pandas as pd
import numpy as np

# Step 2: Load datasets
info = pd.read_csv("students_info.csv")
marks = pd.read_csv("students_marks.csv")

print("Student Info Dataset:")
print(info)
print("\nStudent Marks Dataset:")
print(marks)

# Step 3: Data Cleaning

# Remove duplicate records
info = info.drop_duplicates()

# Handle missing values
info['Age'] = info['Age'].fillna(info['Age'].mean())
marks['Marks'] = marks['Marks'].fillna(marks['Marks'].mean())
```

```

marks['Attendance'] =
marks['Attendance'].fillna(marks['Attendance'].mean())

# Standardize Gender column
info['Gender'] = info['Gender'].replace({'M': 'Male', 'F':
'Female'})

# Step 4: Data Integration
final_data = pd.merge(info, marks, on='Student_ID', how='inner')

print("\nIntegrated Clean Dataset:")
print(final_data)

# Step 5: Save cleaned dataset
final_data.to_csv("cleaned_student_data.csv", index=False)

```

Output (Sample)

Student_ID	Name	Age	Gender	Marks	Attendance
101	Ram	20.0	Male	78.0	85.0
102	Sita	21.0	Female	82.0	85.0
103	Hari	22.0	Male	78.3	90.0

Result

Data preprocessing was successfully performed.

The raw datasets were **cleaned, integrated**, and converted into a **high-quality unified dataset**, suitable for further analysis and data mining.

Conclusion

This lab demonstrated the importance of **data preprocessing** in the data analysis pipeline.

By cleaning and integrating datasets from multiple sources, data quality was improved, ensuring **accurate, reliable, and meaningful analysis**, as required by the KDD process.

ER. SHIVA RAM DAM

Lab 2: Plotting and Inferring Empirical Distribution

Title

Plotting and Inferring Empirical Distribution

Objective

To:

- Understand the concept of empirical distribution based on observed data
- Construct empirical frequency and cumulative distributions
- Plot empirical distributions using histograms and cumulative graphs
- Infer patterns, trends, and variability from empirical data

Software / Tools Required

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries:
 - numpy
 - pandas
 - matplotlib

Theoretical Background (Brief)

An empirical distribution represents the distribution of observed data, without assuming any theoretical model such as normal or exponential distribution.

It shows:

- How frequently values occur
- How data is spread
- Where data is concentrated

Empirical distributions are commonly visualized using:

- Frequency tables
- Histograms
- Empirical Cumulative Distribution Function (ECDF)

This approach reflects what the data actually looks like, not what we assume it should look like.

Dataset Description

Consider the marks obtained by 10 students in a test:

Marks = [45, 50, 55, 60, 60, 65, 70, 75, 80, 85]

Lab Tasks

1. Construct an empirical frequency distribution table
2. Plot a frequency histogram
3. Plot the empirical cumulative distribution function (ECDF)
4. Interpret the distribution and infer patterns

Algorithm / Steps

1. Input the dataset
2. Compute frequency distribution
3. Plot histogram for empirical distribution
4. Compute cumulative frequencies
5. Plot ECDF
6. Analyze the shape and spread of the distribution

Program / Implementation

```
# Step 1: Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Step 2: Input dataset
marks = np.array([45, 50, 55, 60, 60, 65, 70, 75, 80, 85])

# Step 3: Create frequency table
freq_table = pd.DataFrame(marks, columns=["Marks"])
print("Empirical Data:")
print(freq_table)

# Step 4: Plot histogram (Empirical Distribution)
plt.hist(marks, bins=5)
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.title("Empirical Distribution of Marks")
plt.show()

# Step 5: Compute ECDF
sorted_marks = np.sort(marks)
ecdf = np.arange(1, len(sorted_marks)+1) / len(sorted_marks)
```

```
# Step 6: Plot ECDF
plt.step(sorted_marks, ecdf)
plt.xlabel("Marks")
plt.ylabel("Cumulative Probability")
plt.title("Empirical Cumulative Distribution Function (ECDF)")
plt.show()
```

Output (Description)

- The **histogram** shows how marks are distributed across intervals
- The **ECDF plot** shows the proportion of students scoring less than or equal to a given mark

Result

The empirical distribution and ECDF were successfully plotted. The visualizations clearly represent the actual distribution of observed data without assuming any theoretical model.

Inference / Interpretation

- Most students scored between **55 and 75**
- The distribution shows a gradual increase, indicating moderate variability
- About **60% of students scored 65 or below**
- No extreme outliers are observed

Conclusion

This lab demonstrated how empirical distributions help in understanding real-world data.

By plotting histograms and ECDFs, meaningful insights about data concentration, spread, and trends were obtained, forming a foundation for further statistical analysis.

Lab 3: Correlation Analysis

Title:

Correlation Analysis

Objective

To:

- Understand the concept of correlation between variables
- Measure the strength and direction of relationship between two datasets
- Compute correlation coefficients using statistical methods
- Interpret correlation results for data analysis and decision making

Software / Tools Required

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries:
 - numpy
 - pandas
 - matplotlib

Theoretical Background (Brief)

Correlation analysis is used to measure the **degree of relationship** between two variables.

The most commonly used measure is **Pearson's correlation coefficient (r)**:

- $r=+1 \rightarrow$ Perfect positive correlation
- $r=0 \rightarrow$ No correlation
- $r=-1 \rightarrow$ Perfect negative correlation

Correlation helps answer questions such as:

- Does one variable increase when another increases?
- How strong is their relationship?

Note: Correlation does **not** imply causation.

Dataset Description

Consider the following data representing **Study Hours** and **Marks Obtained** by students:

Study Hours (X)	Marks (Y)
1	40
2	45
3	50
4	60
5	65
6	70
7	75

Lab Tasks

1. Plot a scatter diagram to visualize the relationship
2. Compute Pearson correlation coefficient
3. Interpret the correlation result
4. Verify results using built-in functions

Algorithm / Steps

1. Input the datasets
2. Plot scatter plot between two variables
3. Compute mean of both variables
4. Calculate Pearson correlation coefficient
5. Interpret the value of correlation

Program / Implementation

```
# Step 1: Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Step 2: Input datasets
study_hours = np.array([1, 2, 3, 4, 5, 6, 7])
marks = np.array([40, 45, 50, 60, 65, 70, 75])

# Step 3: Scatter plot
plt.scatter(study_hours, marks)
plt.xlabel("Study Hours")
plt.ylabel("Marks Obtained")
plt.title("Scatter Plot: Study Hours vs Marks")
plt.show()
```

```
# Step 4: Compute correlation using numpy
correlation = np.corrcoef(study_hours, marks)[0, 1]
print("Pearson Correlation Coefficient:", correlation)

# Step 5: Compute correlation using pandas
df = pd.DataFrame({
    'Study Hours': study_hours,
    'Marks': marks
})
print("\nCorrelation Matrix:")
print(df.corr())
```

Output (Sample)

- Scatter plot shows an **upward trend**
- Pearson correlation coefficient ≈ 0.98

Result

The correlation coefficient was successfully calculated. A strong positive correlation was observed between **study hours** and **marks obtained**.

Inference / Interpretation

- As study hours increase, marks tend to increase
- The relationship is **strong and positive**
- The variables are highly correlated, but this does not prove causation

Conclusion

This lab demonstrated how correlation analysis helps in understanding relationships between variables. Correlation coefficients and scatter plots provide valuable insights for exploratory data analysis.

Lab 4: Applying and Interpreting Multivariate Linear Regression

Topic

Applying and Interpreting Multivariate Linear Regression

Objectives

- To understand the concept of multivariate linear regression
- To apply multivariate linear regression on a given dataset
- To analyze the effect of multiple independent variables on a dependent variable
- To interpret regression coefficients and model output

Basic Theory (Brief)

Multivariate Linear Regression is a statistical technique used to model the relationship between **one dependent variable** and **two or more independent variables**.

The general form of the regression equation is:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

Where:

- Y = dependent variable
- $X_1, X_2, \dots, X_n$ = independent variables
- β_0 = intercept
- $\beta_1, \beta_2, \dots, \beta_n$ = regression coefficients
- ϵ = error term

Multivariate regression helps in **prediction** and **understanding the influence** of multiple factors simultaneously.

Dataset

Consider the following dataset representing factors affecting student performance:

Study Hours	Attendance (%)	Marks
2	70	45
3	75	50
4	80	60
5	85	65

Study Hours	Attendance (%)	Marks
6	90	70
7	95	78

- Independent Variables: Study Hours, Attendance
- Dependent Variable: Marks

Program Implementation

```
# Step 1: Import required libraries
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Step 2: Create dataset
data = {
    'Study_Hours': [2, 3, 4, 5, 6, 7],
    'Attendance': [70, 75, 80, 85, 90, 95],
    'Marks': [45, 50, 60, 65, 70, 78]
}

df = pd.DataFrame(data)

# Step 3: Separate independent and dependent variables
X = df[['Study_Hours', 'Attendance']]
y = df['Marks']

# Step 4: Create and train the model
model = LinearRegression()
model.fit(X, y)

# Step 5: Display coefficients
print("Intercept:", model.intercept_)
print("Regression Coefficients:")
print("Study Hours:", model.coef_[0])
print("Attendance:", model.coef_[1])

# Step 6: Predict marks
predicted_marks = model.predict(X)
print("\nPredicted Marks:")
print(predicted_marks)
```

Output and Result

- The model successfully fits the data
- Regression coefficients indicate:

- Positive impact of **study hours** on marks
- Positive impact of **attendance** on marks
- Predicted marks are close to actual marks

Sample Output (approximate):

Intercept: -9.85

Study Hours Coefficient: 4.10

Attendance Coefficient: 0.55

Conclusion

Multivariate linear regression was successfully applied to analyze the relationship between multiple independent variables and a dependent variable. The results show that both **study hours** and **attendance** significantly influence student marks.

This technique is useful for prediction and decision-making in real-world data analysis problems.

Lab 5: Non-Parametric Regression using Kernel Methods

Topic

Non-Parametric Regression using Kernel Methods

Objectives

- To understand the concept of non-parametric regression
- To study kernel-based regression methods
- To apply kernel regression on a dataset
- To analyze and interpret the regression output

Basic Theory (Brief)

Non-parametric regression is a regression technique that does **not assume any fixed functional form** between the dependent and independent variables.

Unlike linear regression, kernel regression:

- Learns patterns directly from data
- Is flexible and adapts to nonlinear relationships

A commonly used method is **Kernel Regression**, where predictions are made using a **weighted average of nearby data points**.

The kernel regression estimator is:

$$\hat{y}(x) = \frac{\sum_{i=1}^n K\left(\frac{x-x_i}{h}\right) y_i}{\sum_{i=1}^n K\left(\frac{x-x_i}{h}\right)}$$

Where:

- K = kernel function (e.g., Gaussian kernel)
- h = bandwidth (smoothing parameter)

Dataset

Consider the following dataset representing a **non-linear relationship** between study hours and marks:

Study Hours	Marks
1	30
2	35

Study Hours	Marks
3	50
4	55
5	60
6	65
7	75

- Independent Variable: Study Hours
- Dependent Variable: Marks

Program Implementation

```
# Step 1: Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.kernel_ridge import KernelRidge

# Step 2: Create dataset
X = np.array([1, 2, 3, 4, 5, 6, 7]).reshape(-1, 1)
y = np.array([30, 35, 50, 55, 60, 65, 75])

# Step 3: Create Kernel Regression model
model = KernelRidge(kernel='rbf', alpha=1.0, gamma=0.1)

# Step 4: Train the model
model.fit(X, y)

# Step 5: Predict values
y_pred = model.predict(X)

# Step 6: Plot results
plt.scatter(X, y)
plt.plot(X, y_pred)
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Non-Parametric Regression using Kernel Method")
plt.show()
```

Output and Result

- The kernel regression model successfully fits the dataset
- The predicted curve smoothly follows the nonlinear trend

- The regression line adapts better than linear regression

Conclusion

Non-parametric regression using kernel methods was successfully implemented. The kernel-based model effectively captured the nonlinear relationship between variables without assuming a predefined model. This approach is especially useful when data relationships are complex or unknown.

ER. SHIVA RAM DAM

Lab 6: Principal Component Analysis (PCA)

Topic

Principal Component Analysis (PCA)

Objectives

- To understand the concept of dimensionality reduction
- To apply Principal Component Analysis on a dataset
- To reduce the number of variables while preserving maximum variance
- To interpret principal components and explained variance

Basic Theory (Brief)

Principal Component Analysis (PCA) is a **statistical technique** used for **dimensionality reduction** in multivariate data.

PCA:

- Transforms original correlated variables into a smaller set of **uncorrelated variables** called **principal components**
- Retains maximum information (variance) in fewer dimensions
- Improves visualization, efficiency, and performance of models

Key points:

- First principal component (PC1) captures maximum variance
- Each subsequent component captures remaining variance
- Components are orthogonal to each other

Dataset

Consider a dataset representing student performance:

Study Hours	Attendance	Marks
2	70	45
3	75	50
4	80	60
5	85	65
6	90	70

Study Hours	Attendance	Marks
7	95	78

All variables are numeric and correlated.

Program Implementation

```
# Step 1: Import required libraries
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Step 2: Create dataset
data = {
    'Study_Hours': [2, 3, 4, 5, 6, 7],
    'Attendance': [70, 75, 80, 85, 90, 95],
    'Marks': [45, 50, 60, 65, 70, 78]
}

df = pd.DataFrame(data)

# Step 3: Standardize the data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)

# Step 4: Apply PCA
pca = PCA(n_components=2)
principal_components = pca.fit_transform(scaled_data)

# Step 5: Display results
print("Principal Components:")
print(principal_components)

print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)
```

Output and Result

- Dataset successfully transformed into **two principal components**
- First principal component captures the **maximum variance**
- Explained variance ratio indicates how much information is retained

Sample Output (approximate):

Explained Variance Ratio:

[0.92, 0.07]

This means:

- PC1 captures about **92%** of total variance
- PC2 captures about **7%** of total variance

Conclusion

Principal Component Analysis was successfully applied to reduce dimensionality of the dataset.

The transformation preserved most of the data variance using fewer variables. PCA is useful for data visualization, noise reduction, and improving model performance in high-dimensional data.

Lab 7: Decision Trees for Classification using Information Gain, Gain Ratio, and Gini Index

Topic

Decision Trees for Classification using Information Gain, and Gini Index

Objectives

- To understand and implement decision tree classification.
- To construct decision trees using:
 - Information Gain (ID3 algorithm)
 - Gain Ratio (C4.5 algorithm)
 - Gini Index (CART algorithm)
- To classify new data based on the constructed decision tree

Software/Tools Required

- Python 3.x
- Libraries: pandas, numpy, scikit-learn, matplotlib (optional for visualization)
- Jupyter Notebook or any Python IDE

Basic Theory (Brief)

A **Decision Tree** is a supervised learning algorithm used for **classification and prediction**.

It represents decisions in the form of a **tree structure**, where:

- Internal nodes represent tests on attributes
- Branches represent outcomes of the test
- Leaf nodes represent class labels

To select the best attribute at each split, the following criteria are used:

1. Information Gain (IG)

- Based on **entropy**, a measure of impurity
- Chooses the attribute that gives maximum reduction in entropy

2. Gain Ratio

- Improves Information Gain by reducing bias toward attributes with many values
- Uses **split information** for normalization

3. Gini Index

- Measures the impurity of a dataset
- Lower Gini Index indicates a better split
- Commonly used in CART decision trees

Dataset

Consider a dataset for predicting whether there will be Play or not based on weather information:

- Independent Variables: Temp, Humidity, Wind
- Dependent Variable: Play

Training data:

Temp	Humidity	Wind	Play
Hot	High	Weak	No
Hot	High	Strong	No
Mild	High	Weak	Yes
Cool	Normal	Weak	Yes
Cool	Normal	Strong	No

Test data:

Temp	Humidity	Wind	Play
Mild	High	Strong	?

Program Implementation

Step 1: Load the Data

```
import pandas as pd

# Sample dataset
data = {
    'Temp': ['Hot', 'Hot', 'Mild', 'Cool', 'Cool'],
    'Humidity': ['High', 'High', 'High', 'Normal', 'Normal'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Strong'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'No']
}
```

```
}

df = pd.DataFrame(data)
df
```

Step 2: Decision Tree Using Information Gain (ID3)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

# Features and target
X = pd.get_dummies(df[['Temp', 'Humidity', 'Wind']])
y = df['Play']

# ID3 equivalent using entropy
clf_id3 = DecisionTreeClassifier(criterion='entropy',
                                random_state=0)
clf_id3.fit(X, y)

# Visualize the tree
tree.plot_tree(clf_id3, feature_names=X.columns,
               class_names=clf_id3.classes_, filled=True)
```

Step 3: Decision Tree Using Gain Ratio (C4.5)

```
# For demonstration, we'll use the same entropy tree as C4.5 proxy
clf_c45 = DecisionTreeClassifier(criterion='entropy',
                                random_state=0)
clf_c45.fit(X, y)
```

Step 4: Decision Tree Using Gini Index (CART)

```
# Gini criterion
clf_cart = DecisionTreeClassifier(criterion='gini', random_state=0)
clf_cart.fit(X, y)

# Visualize the tree
tree.plot_tree(clf_cart, feature_names=X.columns,
               class_names=clf_cart.classes_, filled=True)
```

Step 5: Classifying New Data

```
# New example to predict
new_data = pd.DataFrame({
    'Temp': ['Mild'],
    'Humidity': ['High'],
```

```

        'Wind': ['Strong']
    })

new_data_encoded = pd.get_dummies(new_data)
# Align with training data columns
new_data_encoded = new_data_encoded.reindex(columns=X.columns,
fill_value=0)

# Predictions
print("ID3 Prediction:", clf_id3.predict(new_data_encoded)[0])
print("CART Prediction:", clf_cart.predict(new_data_encoded)[0])

```

Output and Result

- Decision tree models were successfully trained
- Attribute splits were selected based on impurity reduction
- Both models classified the dataset correctly

Observations:

- **Information Gain** focuses on entropy reduction
- **Gini Index** provides faster computation and simpler splits

Conclusion

Decision Trees for classification were successfully implemented using different attribute selection measures.

Information Gain, Gain Ratio, and Gini Index help in selecting optimal splits during tree construction.

Decision trees are easy to interpret and effective for classification tasks, especially when decision rules are required.

Lab 8: Logistic Regression

Title:

Logistic Regression for Binary Classification

Objective:

- Understand the concept and working of logistic regression.
- Implement logistic regression in Python to classify binary outcomes.
- Evaluate the model using accuracy, confusion matrix, and ROC curve.

Basic Theory:

- Logistic Regression is a **supervised learning algorithm** used for **classification**, particularly **binary classification**.
- Unlike linear regression, which predicts continuous values, logistic regression predicts **probabilities** between 0 and 1 using the **sigmoid function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Where $z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$
- **Decision rule:** If probability $\geq 0.5 \rightarrow$ class = 1, else class = 0
- Applications: Predicting disease (Yes/No), customer churn, survival, etc.

Dataset:

We will use the **Titanic dataset** (available in Kaggle or as titanic.csv)

Columns used for this lab:

- Age – numeric
- Fare – numeric
- Sex – categorical (Male/Female)
- Pclass – categorical (1, 2, 3)
- Embarked – categorical (C, Q, S)
- Survived – target (0 = No, 1 = Yes)

Programs:

Step 1: Import Libraries

```
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report, roc_curve, roc_auc_score
```

Step 2: Load Dataset

```
data = pd.read_csv('titanic.csv')
data.head()
```

Step 3: Data Preprocessing

```
# Handle missing values
data['Age'].fillna(data['Age'].mean(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

# Convert categorical variables into numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
data = pd.get_dummies(data, columns=['Embarked', 'Pclass'],
drop_first=True)

# Features and target
X = data[['Age', 'Fare', 'Sex', 'Embarked_Q', 'Embarked_S',
'Pclass_2', 'Pclass_3']]
y = data['Survived']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 4: Build Logistic Regression Model

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

Step 5: Predictions

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
```

Step 6: Evaluate Model

```

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Classification Report
print("Classification Report:\n", classification_report(y_test,
y_pred))

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, color='blue', label='ROC curve (area = %0.2f)'
% roc_auc)
plt.plot([0,1], [0,1], color='red', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc="lower right")
plt.show()

```

Result and Analysis:

- **Confusion Matrix:** Shows True Positives, True Negatives, False Positives, False Negatives.
- **Accuracy:** e.g., Accuracy: 0.81 → the model correctly predicts survival 81% of the time.
- **Classification Report:** Precision, Recall, F1-score for each class.
- **ROC Curve and AUC:** The closer the curve to the top-left, the better the model; AUC indicates model discrimination ability (1 = perfect).
- **Observations:** Logistic Regression works well for linearly separable features; scaling improves performance.

Conclusion:

- Logistic Regression is an effective and simple algorithm for **binary classification**.
- Feature preprocessing, encoding, and scaling are crucial for better results.

- Model evaluation using **accuracy, confusion matrix, and ROC-AUC** helps assess performance.
- It is widely used in healthcare, finance, marketing, and social sciences for prediction tasks.

ER. SHIVA RAM DAM

Lab 9: ARIMA Model for time series analysis

Title:

Time Series Forecasting using ARMA and ARIMA Models

Objective:

- Understand ARMA and ARIMA models for time series analysis.
- Apply ARMA to stationary data and ARIMA to non-stationary data.
- Forecast future values and evaluate model performance.

Basic Theory:

- **ARMA (p,q)** combines:
 - **AR(p)**: current value depends on past p values.
 - **MA(q)**: current value depends on past q forecast errors.
 - Assumes stationary time series.
- **ARIMA (p,d,q)** extends ARMA for non-stationary data:
 - **I (Integrated)**: differencing to make the series stationary.
 - Useful for trends or changing patterns in time series.
- Steps include checking stationarity, selecting lag values, model fitting, and forecasting.
- Lag selection often uses **AIC/BIC** criteria to balance fit and simplicity

Dataset:

- Example dataset: Monthly sales data (can be replaced with any univariate time series).
- Month: Jan, Feb, Mar, Apr, May, Jun
- Sales: 120, 130, 125, 140, 135, 150

Program Implementation (Python):

```
import pandas as pd
import numpy as np
from statsmodels.tsa.arima.model import ARIMA
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# Dataset
data = [120, 130, 125, 140, 135, 150]
ts = pd.Series(data)

# Plot time series
plt.plot(ts)
plt.title('Time Series Data')
```

```
plt.show()

# Check stationarity (visual + differencing if needed)
ts_diff = ts.diff().dropna()

# Plot ACF and PACF
plot_acf(ts_diff)
plot_pacf(ts_diff)
plt.show()

# Fit ARIMA model (example: ARIMA(1,1,1))
model = ARIMA(ts, order=(1,1,1))
model_fit = model.fit()
print(model_fit.summary())

# Forecast next 3 values
forecast = model_fit.forecast(steps=3)
print("Forecasted values:", forecast)
```

Result and Analysis:

- The ARIMA model summary shows coefficients for AR, MA terms.
- Forecasted values predict future sales/trends.
- Plotting ACF and PACF helps understand autocorrelations and select proper lag orders.
- Analysis ensures model captures patterns without overfitting.

Conclusion:

- ARMA works for stationary series; ARIMA is better for non-stationary data with trends.
- Correct differencing and lag selection improve forecast accuracy.
- Time series forecasting is essential for financial, industrial, and meteorological predictions.

Lab 10: Granger Causal Inference

1. Objective:

- To understand and apply Granger causality to test predictive relationships between time series variables.
- To differentiate between correlation and predictive causality in time series data.
- To interpret the results of Granger causality tests for practical applications.

2. Basic Theory:

- **Causation vs Correlation:** Correlation shows association; causation implies one variable influences another. Granger causality is a statistical approach to assess if past values of one time series improve prediction of another.
- **Granger Causality:** If past values of variable X help predict future values of variable Y better than past values of Y alone, X is said to Granger-cause Y.
- **Key Points:**
 - Does **not prove true causation**, only predictive causality.
 - Requires stationary time series.
 - Typically uses lagged values determined by criteria such as AIC or BIC.

3. Dataset:

- Example: Monthly advertising spending (X) and monthly sales (Y) for 3 years.
- Dataset should be equally spaced, cleaned, and preprocessed for missing values.

4. Program Implementation (Python Example):

```
# Import libraries
import pandas as pd
from statsmodels.tsa.stattools import grangercausalitytests
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv('monthly_sales_advertising.csv')
data['Month'] = pd.to_datetime(data['Month'])
data.set_index('Month', inplace=True)

# Visualize the time series
data.plot(figsize=(10,5))
plt.title('Monthly Advertising Spending vs Sales')
plt.show()

# Perform Granger causality test
# Null hypothesis: X does not Granger-cause Y
max_lag = 3
```

```
granger_test = grangercausalitytests(data[['Sales',
'Advertising']], maxlag=max_lag, verbose=True)
```

5. Result and Analysis:

- **Output Interpretation:**
 - If p-value < 0.05 for a lag, reject the null hypothesis: X Granger-causes Y.
 - Compare F-statistics for different lags to determine strongest predictive relationship.
- **Example Outcome:**
 - Lag 1: p-value = 0.08 → Not significant
 - Lag 2: p-value = 0.03 → Significant → Advertising spending helps predict sales
- **Analysis:**
 - Advertising Granger-causes sales at lag 2.
 - Helps in forecasting and strategic planning but does not imply direct causation.

6. Conclusion:

- Granger causality is a powerful tool for testing predictive causality in time series.
- Past values of one variable can enhance prediction of another variable.
- Stationarity and proper lag selection are critical for accurate results.
- Useful in economics, finance, and marketing analytics for forecasting decisions
